Visual Object Tracking using Adaptive Correlation Filter

Azadeh Hadadi[1], Jue Wang[1], Dr. Marc Blanchon[2]
[1]Condorcet University Centert, Universite Bourgogne et Franche-Compte, 71200 Le Creusot, France
2 Laboratoire Électronique Informatique Image (LE2I)

Although the traditional template matching is considered as a tracker and achieves some results, it fails in the presence of constraints and in terms of both on accuracy and robustness there is a need to improve the overall tracking capability. This paper focuses on visual object tracking based on the Adaptive Correlation Filter method introduced in the research paper of Bolme et al. [1]. They derived a fast correlation filter, i.e., MOSSE tracker, which utilizes classical signal processing analysis and contains a minimum output sum of squared errors filter. This application is an implementation of this approach and has the ability to track multiple objects simultaneously. It has implemented in Python 3.7 which uses OpenCV library. The result of the current implementation based on [1] will be compared with the cross-correlation base technique. These comparisons will be plotted in MATLAB.

***Index Terms*—Visual Object Tracking, Adaptive Correlation Filter, MOSSE, Template Matching, Cross-Correlation**

I. Introduction

One of the fundamental research problem in computer vision community is visual tracking due to its wide range of applications in video surveillance, robotics, human-computer interaction and driverless vehicle.

Theoretically, visual tracking is the process of locating a moving object (or multiple objects) over time using a camera. It is the core of a vision-based control system in robotics (Hutchinson et al. [2]). Based on measurements, visual tracking methods can be classified into two main groups:

- Methods that track local features such as line segments, edges or contours across the sequence (Isard and Blake [3], Torr and Zisserman [4], Drummond and Cipolla [5]) which are sensitive to feature detection
- Methods that only make use of image intensity information, estimate the movement, the deformation or the illumination parameters of a reference template between two frames by minimizing an error measure based on image brightness

Accuracy and robustness are two criteria to measure the performance of trackers. Recently researchers have focused on improving robustness, using adaptive models. The purpose of the paper is to briefly review the Adaptive Correlation Filter algorithm and show how it can be implemented to do this end. One of the traditional methods for tracking was to convolve the template with a target frame and checking for a strong 2D Gaussian shaped peak centered on the target in the output image. Although it has worked in some cases, they prone to errors because of the complexity of the target object model, since the model depends on a lot of environmental conditions such as lighting, occlusion, rotation, scaling and whatsoever.

These problems are eliminated by Adaptive Correlation Filters approach which updates the filter in real-time, it does so by finding a filter that minimizes the sum of squared error between the actual output of the correlation and the desired output of the correlation. This algorithm is called Minimum Output Sum of Squared Error (MOSSE). It trains filtering algorithm relying only on one image. The overall concept of the MOSSE algorithm is shown in Figure 1 . In the first frame, a patch of image is selected which contains a target object. The idea is to track this object in the following frame.

Manuscript submited Dec 12, 2020 for evaluation. Corresponding authors: Azadeh Hadadi, Jue Wang.

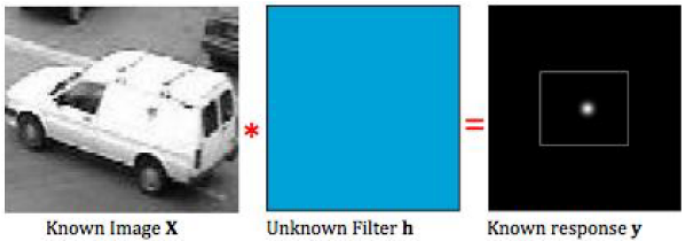

Fig. 1. MOSSE concept

The remaining part of the report is organized as follows: In section II literatures of the related works will be reviewed and summarized. The MOSSE algorithm is introduced in section III. Moreover, the mathematical equations to drive the adaptive filter and performance measure (PSR) are detailed in this section. Section IV will be dedicated to implementation of the method. Results will be presented in section V and discussed.The paper will end up to conclusion and references.

II. Review and Related Works

Trackers which work based on correlation filter have been proven to be competitive with far more complicated tracking methods. Tracking approach with the CF has attracted a great number of attention because of its high computational efficiency, accuracy, and robustness. Henriques et al. [6] designed kernel classifiers with the same characteristics as correlation filters, i.e., CSK tracker. The kernelized least-squares classifier in the CSK tracker can be trained and evaluated by fast Fourier transform. Based on the framework of CSK tracker, Danelljan et al. [7] introduced color attributes and proposed an adaptive dimension reduction technique, which can preserve useful information while greatly reducing the number of color dimensions. In [8], Henriques et al. combined techniques of kernel trick and circulant matrix to train kernelized correlation

Fig. 2. perturbation processing for creating training image database

filter (KCF), i.e., KCF tracker, with HOG feature. The KCF tracker has more adaptive performance for different scenarios by using multi-channel HOG feature. Moreover, Li et al. proposed a scale adaptive kernel correlation filter, i.e., SAMF tracker [9], which integrates the HOG and CN [10] features, thereby learning a model that is inherently robust to both deformations and illumination changes. However, the synthetic negative samples, which are generated by the circular shift over the object, are used in the filter training stage. In addition, the aforementioned trackers have the boundary effect problem in the training and detection stages, which significantly influences the tracking performance.

III. MOSSE Algorithm and Theory

The main objective of MOSSE algorithm is to show how filter is calculated and updated for a new frame. MOSSE's filter is similar to Average of Synthetic Exact Filters (ASEF) , but using less number of training images. Meanwhile, the result of MOSSE filter is more robust than ASEF filter [11]. It is also faster when training the image. MOSSE filter also produces stable correlation filters when using single frame initialization.

In fact, a given target window is perturbed by shifting the target to different direction and making training images. Figure 2 shows the perturbation process and its parameters. In this method only one of the images contains target object and the rest of the images does not contain. The response image for the image containing target is a gaussian image with the peak at the center of the image and for the rest of the image the response is zero. Figure 3 shows an example of training image database outputted from perturbation process.

Now, we have a set of training images and their response as is displayed in Figure 3, the question is "which filter matches the training image to the responses?" one of the answer proposed by the author to this question is to use linear least square optimization. However, before applying this method, x images and y responses are transformed to the ***Frequency domain*** because the calculations will be simpler. In summary, the target of this optimization is find h filter so as to optimize one of the following cost function Eq. 1, 2.

$$E(\mathbf{h}) = \frac{1}{2}\sum_{\tau\in\mathcal{C}} \left\| y_\tau - \mathbf{x}[\tau]^T\mathbf{h} \right\|_2^2 \tag{1}$$

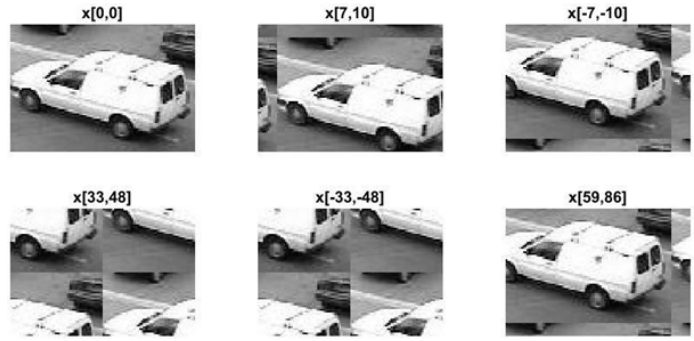

Fig. 3. training image example generated in the perturbation process

$$E(\mathbf{h}) = \frac{1}{2}\sum_{\tau\in\mathcal{C}} \left\| y_\tau - \mathbf{x}[\tau]^T\mathbf{h} \right\|_2^2 + \frac{\lambda}{2}\|\mathbf{h}\|_2^2 \tag{2}$$

Wide variety solutions stems from different approaches have suggested for the above optimization problem. Following section explains how h filter can be derived given the perturbed target image patches and their response.

A. Mathematical Equations to Derive Adaptive Filter

The expectation of the filter applying on the input image is maximum response (highly correlated). As was mentioned in previous section, to create a fast tracker, correlation is computed in Fourier domain. Based on Convolution theorem, correlation becomes an element-wise multiplication in Fourier domain. The input image is considered as f and h as the filter. Fourier transform of them are calculated as F and H respectively:

$$\begin{aligned} F &= \mathcal{F}(f) \\ H &= \mathcal{F}(h) \end{aligned} \tag{3}$$

If the output is considered as g (we assume it is Gaussian), the desired output is the correlation of input image and filter:

$$\mathcal{F}(g) = G = F \odot H^* \tag{4}$$

The entire computation process has an upper-bound time of $O(P \log P)$ where P is the number of pixels in the tracking window. If f_i and g_i are the set of training images and the set of training outputs, respectively, their Fourier transforms become F_i and G_i :

$$H_i^* = \frac{G_i}{F_i} \tag{5}$$

So, the filter H should be fined that minimizes the sum of squared error between the actual output of the correlation and the desired output of the correlation. Hence this filter is also known as ***MOSSE*** filter (minimum output sum of squared error).

$$\min_{H^*}\left(\sum_i |F_i \times H^* - G_i|^2\right) \tag{6}$$

$$\frac{\partial}{\partial H^*}\left(\sum_i |F_i \times H^* - G_i|^2\right) = 0 \tag{7}$$

$$\frac{\partial}{\partial H^*}\left(\sum_i (F_i \times H^* - G_i)(F_i \times H^* - G_i)^*\right) = 0 \tag{8}$$

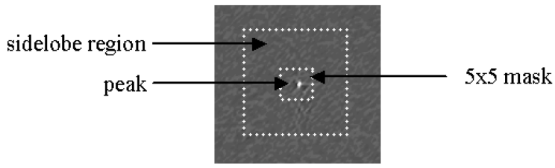

Fig. 4. PSR estimiation

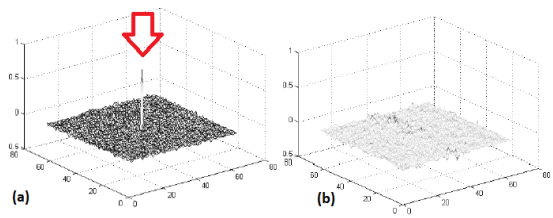

Fig. 5. sample correlation plane for test image from (a) true-class, PSR = 37 (b) false class , PSR=3

$$\sum_i (F_i F_i^* H - F_i G_i^*) = 0 \tag{9}$$

$$H = \frac{\sum_i (F_i G_i^*)}{\sum_i (F_i F_i^*)} = \frac{\sum_i (F_i G_i^*)}{\sum_i (I_i)} \tag{10}$$

$$H^* = \frac{\sum_i (F_i^* G_i)}{\sum_i (F_i F_i^*)} \tag{11}$$

In Eq. 11, the numerator is the correlation between the input and the desired output and the denominator is the intensity of the input. There is a new factor called ***Learning rate*** (η) , learning rate puts more weight on recent frames and lets the effect of previous frames decay exponentially over time. It helps the filter adapt to the changes of the object.

$$H^* = \frac{A_i}{B_i} \tag{12}$$

$$A_i = \eta F_i^* G_i + (1-\eta) A_{i-1} \tag{13}$$

$$B_i = \eta F_i^* F_i + (1-\eta) B_{i-1} \tag{14}$$

B. Performance Measure

As evaluation, cross-correlation is performed on frames. For authentic, the correlation output should be sharply peaked and it should not exhibit such strong peaks for frames that are not match with the template. So, to measure the peak sharpness and performance, ***Peak Sidelobe Ratio (PSR)*** is used. In fact, the PSR is the ratio between the returned signal of the mainlobe and that of the first sidelobe (local maxima) of the point signal. This parameter is also expressed in decibels.

How PSR is estimated? First, the peak is located (shown as the bright pixel in the center of the Figure 5). The mean and the standard deviation of the e.g. 20x20 sidelobe region (excluding a 5x5 central mask) centered at the peak are computed. Occlusion can be detected by setting a threshold value which helps in freezing the tracker when the object is covered and resume tracking when the object reappears. Mathematically, PSR is defined as Eq. 15:

$$PSR = \frac{peak - \mu}{\sigma} \tag{15}$$

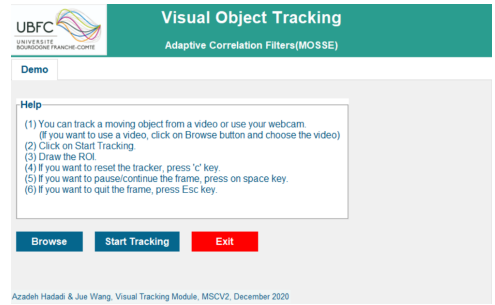

Fig. 6. application GUI

Fig. 7. sample result of the MOSSE filter

Where μ and σ are the mean and standard deviation of the sidelobe.

In Figure 5 a typical correlation output is displayed. Figure 5 (a) shows a sharp correlation peak resulting in a large PSR value of 37. The next correlation output in Figure 5(b) shows a typical response to false class exhibiting low PSRs (<10).

IV. IMPLEMENTATION

The code implemented by Python 3.7 and used OpenCV library. A GUI has developed for this project to get easy access to user and helps him how to use the application as is shown in Figure 6. There are two options: choosing an external video as a source or use the captured video from the webcam directly.

(For the first choice, ***Browse*** button should be clicked at first and selected the desired video and then followed the next steps). ***Start Tracking*** button will run the tracker and should be pressed. If any video was not selected by the user, it will turn on the webcam.In both modes, it shows the desired video. User should draw the ROI around the target object. ***Exit*** button closes the application.

There are two main classes in the project: ***Controller*** and ***MOSS***E. The role of Controller class is showing the desired video, allowing drawing ROI on the frame and receiving its coordinates, drawing a rectangle around the target object in the video stream based on the result of MOSSE along with the PSR value at the bottom left corner of the region (Figure 7), and showing a window composed of ***input image of target, related filter and correlation output*** (Figure 8). This class will run the MOSSE algorithm.

MOSSE class receives ROI and then builds the gaussian kernel. It builds the kernel based on the center of the target object position and its same size to be focused on only in the image patch. As mentioned in the paper, MOSSE filter applies

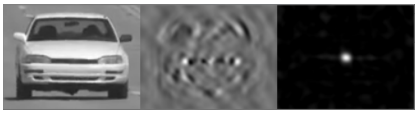

Fig. 8. input, filters and correlation output

random small affine perturbations to the tracking window for the first image patch in order to produce the better result of filtering. It is implemented in ***rnd_warp*** method. Then some preprocessing image patch will be applied to reduce the topological torus structure from mapping image and filter. In this part there are 3 steps to do:

- Apply log function to help low contrast lighting situations
- Normalized pixel values:
  image = (image - imgage.mean) / (image.std + eps)
- Multiply the image by cosine window. In this code implementation, Hann window is applied since ***Hanning function*** implements the cosine function.

Next step is creating training set (f_i) from original image and get the output (g_i). The output is generated from ground-truth such that it has a compact 2D gaussian shaped peak centered on the target in training image f_i. The image patch has to be converted into Fourier transformation to do the next process, since all process for MOSSE filter is occurred in Fourier domain. Then the gist of MOSSE algorithm, i.e. calculation of the Fourier Transform of input image path and correlation between it and Gaussian kernel is done according to Eq. 11.

Kernel and images in sequence will be updated. In ***update*** method, at first preprocessing will be applied on frames. Then the correlation between image and kernel, finding the maximum value of the correlation result (g_i) (the maximum value is the strongest peak) and calculating the PSR measurement are done in ***correlation*** method.

8.0 is considered as PSR threshold value. For every image in the image sequence, add learning rate (η) that allows filter to quickly adapt to appearance changes while still maintaining a robust kernel. The default value of learning rate is***0.125*** (Eq. 13, 14). Then, the new patch will be created, it is moving from the old coordinate to the new center coordinate based on ***dx*** and ***dy***. Cropping the new patch based on the new coordinate center and updating the A and H values with the different weight value (learning rate = η) for the previous A and current A.

If the PSR value meets the threshold, Controller will draw a circle in the middle of the green rectangle. Otherwise, two cross-over lines will be displayed on the red rectangle.

The possibility to track multiple objects applied in this application. If the user tracks more than one object, the last input image of target, related filter and correlation output is displayed.

As mentioned in the help of GUI, the user can control the process by keyboard:

- ***Space***: Pause/Continue the video
- ***Esc***: Quit the vide and frames
- ***C***: Clear trackers list

Fig. 9. ground truth

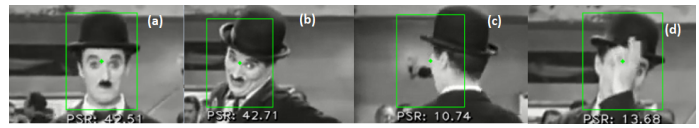

Fig. 10. Good performance in the presence of occlusion and rotation (high PSR)

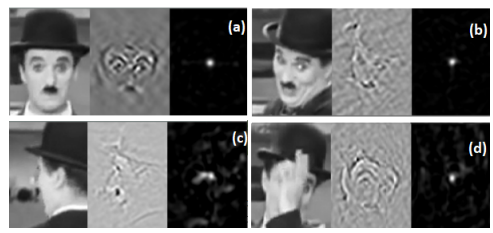

Fig. 11. input image, kernel and correlation output in high PSR

V. RESULT AND DISCUSSION

MOSSE filter applied on the some videos. This method works based on a target model. Figure 9 shows the target model (ground truth). The results confirm that this method can track in the presence of occlusion, and off-plane rotation.

In Figure 10 four sample images are displayed which the first one is perfectly matches with the the ground truth (ROI). Frames (b), (c) and (d) are related to rotation, off-plane rotation and occlusion respectively. As shown in the figure, PSR value in (d) and (c) is less than (b). However, it meets the threshold value, i.e. 8.0, and they are considered as good state.

Figure 11 presents the kernel and correlation output belongs to image patches in previous figure. As is expected from the PSR values, path (b) has a sharper peak than (d) and (c).

Correlation responses are shown in 3D states in Figure 12. (a) has the sharpest peak which is matched with the ground truth completely. Between (b) and (d), the neighborhood is more highlighted in (d) and its peak is not as lofty as (b). The worst one is related to out-off plane (c). As is displayed in the figure, there is not a unique peak and the neighborhood even is more highlighted than (d). However, MOSSE could track the target and PSR is greater than the threshold. It shows the good performance of algorithm in tracking the object.

Figure 13 shows a sample of low PSR. In this frame, the target could not been tracked by MOSSE and PSR does not meet the threshold. ($5.79 < 8$). Correlation output in Figure 13 confirms this analysis. We do not have a unique sharp peak and also the adjacent objects in the vicinity of the patch of the moving object are as calling attention to as the peaks.

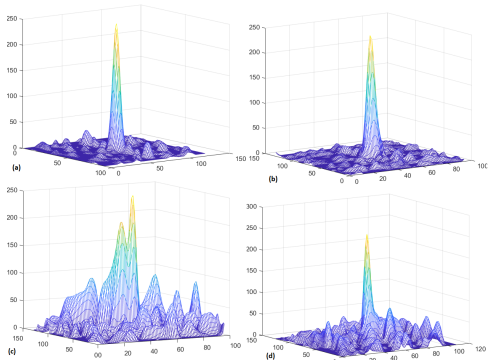

Fig. 12. response to the target location in high PSR

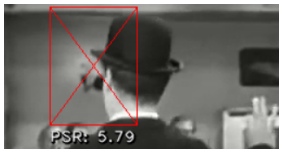

Fig. 13. sample of low PSR

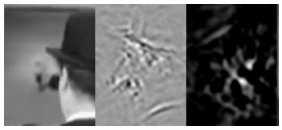

Fig. 14. input image, kernel and correlation output of low PSR

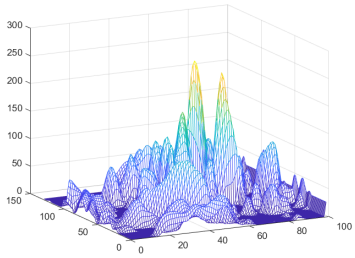

Fig. 15. response to the location target in low PSR

In this part, ordinary cross_correlation outputs are compared with MOSSE results. Six traditional template matching methods, i.e.

- TM_SQDIFF
- TM_CCOEFF
- TM_CCOEFF_NORMED
- TM_CCORR
- TM_CCORR_NORMED
- TM_SQDIFF_NORMED

were applied on desired frames and matched with template patch (ground truth). TM_SQDIFF results as an example are shown in Figure 16 and Figure 17. The results show that just in the case that there is not any constraint and the template is clearly visible we have a sharp peak in the output correlation. Otherwise, it will fail.

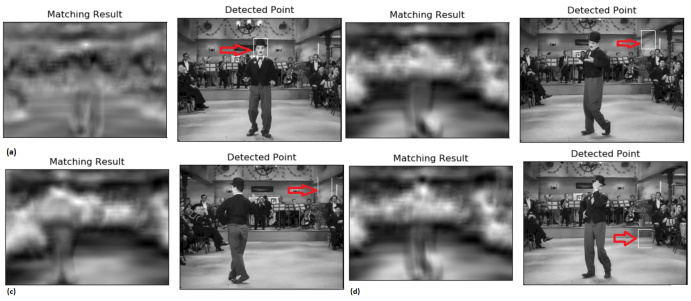

Fig. 16. template matching using TM_SQDIFF method

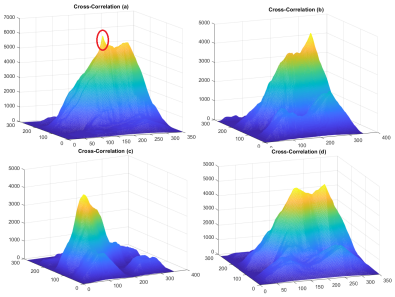

Fig. 17. ordinary cross_correlation output on selected image patches

VI. Conclusion

The results obtained in this implementation shows that using an adaptive training scheme, MOSSE is considerably robust and efficient in tracking. In other words when the object target encounters occlusion, off-plane, and rotation yet the tracker still can work quite well (unlike the traditional template matching which can work just in cases without any constraints and fails to produce a peak in case of occlusion and off-plane rotation) and fast since the main idea of applying MOSSE filter is to be faster in tracking an object as well as keep the robustness of the filter to the object. Based on experimental results, tracking using adaptive correlation filters, which using MOSSE filter are working well and fast.

References

[1] D. S. Bolme, J. R. Beveridge, B. A. Draper, and Y. M. Lui, "Visual object tracking using adaptive correlation filters," in *2010 IEEE computer society conference on computer vision and pattern recognition*. IEEE, 2010, pp. 2544–2550.

[2] S. Hutchinson, G. D. Hager, and P. I. Corke, "A tutorial on visual servo control," *IEEE transactions on robotics and automation*, vol. 12, no. 5, pp. 651–670, 1996.

[3] M. Isard and A. Blake, "Contour tracking by stochastic propagation of conditional density," in *European conference on computer vision*. Springer, 1996, pp. 343–356.

[4] P. H. Torr and A. Zisserman, "Feature based methods for structure and motion estimation," in *International workshop on vision algorithms*. Springer, 1999, pp. 278–294.

[5] T. Drummond and R. Cipolla, "Visual tracking and control using lie algebras," in *Proceedings. 1999 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (Cat. No PR00149)*, vol. 2. IEEE, 1999, pp. 652–657.

[6] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, "Exploiting the circulant structure of tracking-by-detection with kernels," in *European conference on computer vision*. Springer, 2012, pp. 702–715.

[7] M. Danelljan, F. Shahbaz Khan, M. Felsberg, and J. Van de Weijer, "Adaptive color attributes for real-time visual tracking," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 1090–1097.

[8] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, "High-speed tracking with kernelized correlation filters," *IEEE transactions on pattern analysis and machine intelligence*, vol. 37, no. 3, pp. 583–596, 2014.

[9] Y. Li and J. Zhu, "A scale adaptive kernel correlation filter tracker with feature integration," in *European conference on computer vision*. Springer, 2014, pp. 254–265.

[10] J. Van De Weijer, C. Schmid, J. Verbeek, and D. Larlus, "Learning color names for real-world applications," *IEEE Transactions on Image Processing*, vol. 18, no. 7, pp. 1512–1523, 2009.

[11] Z. Chen, Z. Hong, and D. Tao, "An experimental survey on correlation filter-based tracking," *arXiv preprint arXiv:1509.05520*, 2015.